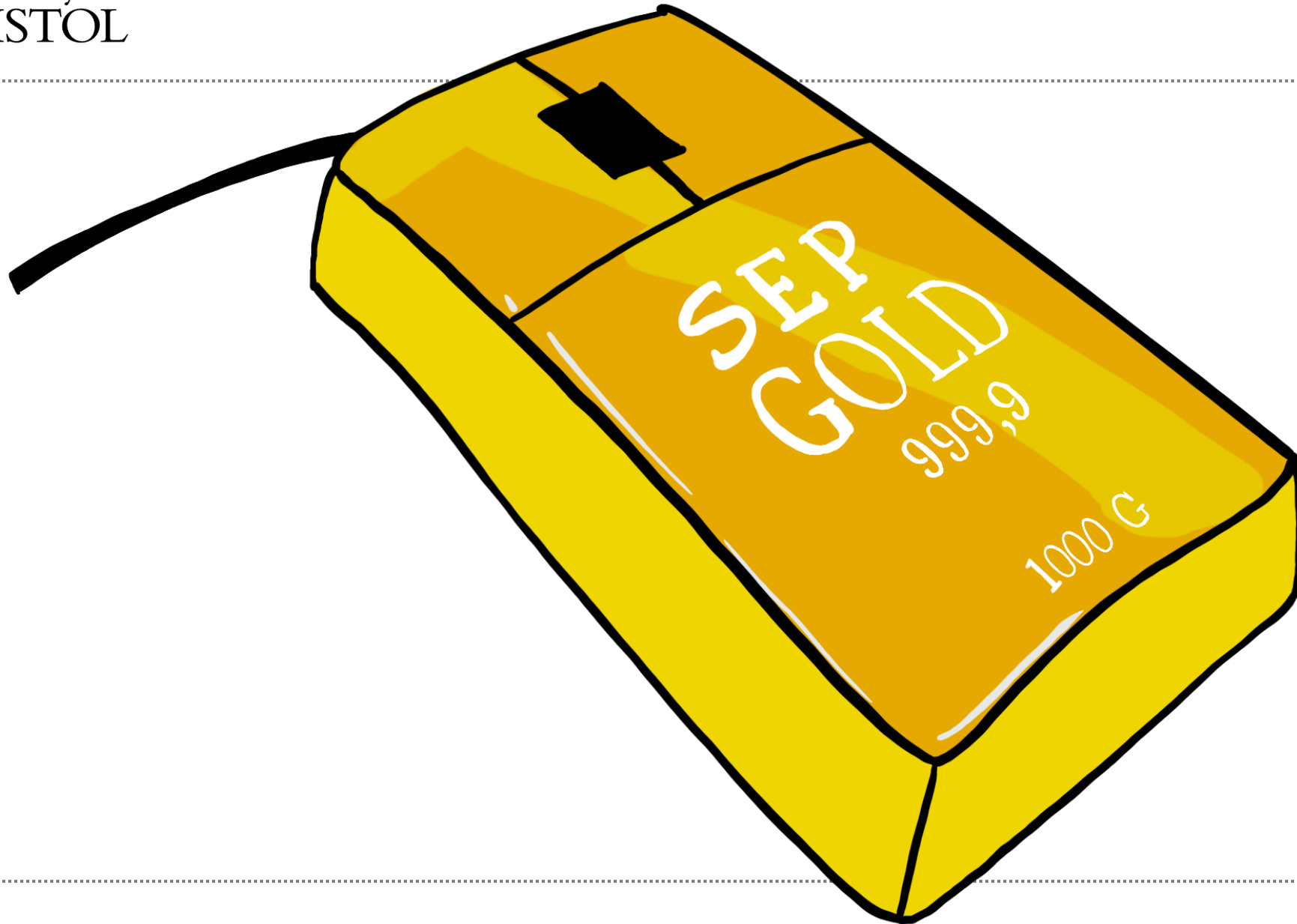




SEP Week 2

Software Processes

(thanks to Simon Lock, whose slides I've adapted for some of this lecture)

Learning about basics of software development processes...

... So you can meet your team tomorrow, and start work!

Developing software is a bit like cooking:

Chefs (Developers)

follow a recipe (process)

To cook a meal for hungry diners (clients)

Up until now, you've mostly been 'freestyling'
(cooking without a recipe)

Creating meals for yourself
Or to feed friends and family (lecturers!)

But now you need to cook 'commercially'
For paying customers

For the best chance of success, we need

Talented chefs

(who can follow a clearly defined recipe and work together)

A clear recipe

(that defines what we should be doing when)

Decisive diners

(who can make up their minds what they want to eat!)

Different kinds of restaurants

Different restaurants run different types of kitchen

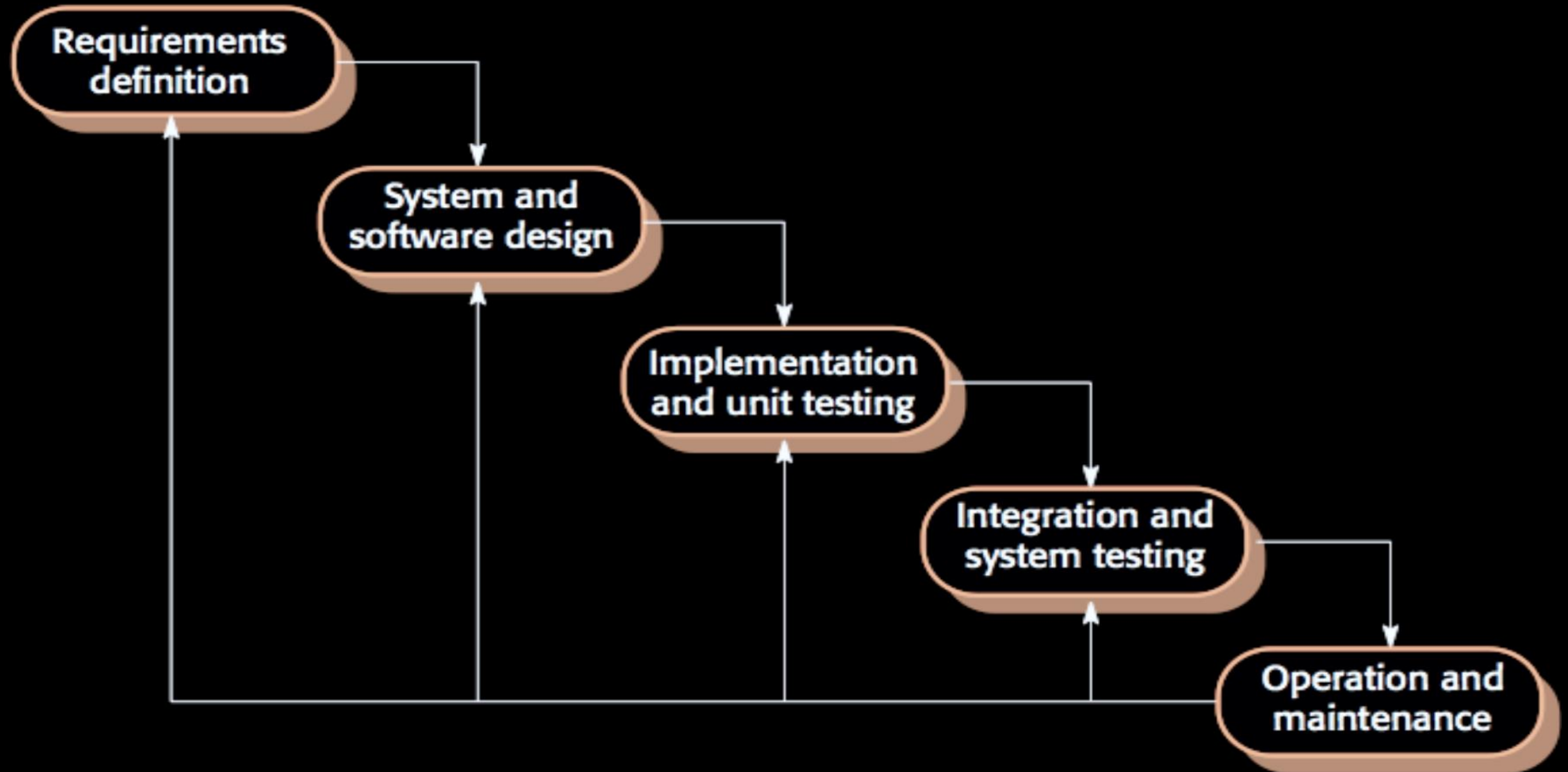
- You won't expect a burger bar to cook in the same way as a high-end restaurant
- Similarly, developers build in different ways for different products and markets
- You won't expect an autonomous car to be built in the same way as a social media app!

- Mobile productivity apps
- Stand-alone desktop applications
- Transaction-based networked applications
- Embedded control systems
- Data collection and monitoring systems
- Batch processing systems
- Entertainment and education apps
- Systems for modelling and simulation
- Games
- Etc

“THE” Software Development process

- No such thing as "THE" software dev process!
- Different domains rely upon different processes
- Different companies have own in-house standards
- Similar concepts can have different names
- Different experts advocate their own approach
- All will try and convince you that theirs is the best!

- There is limited agreement/standardisation
- We'll try to put across something coherent
- Present some key concepts and processes
- Might differ from what your client advocates
- Might differ from what you do when you graduate
- But majority of the fundamentals you will recognise



- What should the system do?
- What are the needs of the users?
- What are the needs of the host organisation?
- What are interoperability needs of existing systems?

Requirements

We need to think about functional elements
(stuff that system should do)

And also non-functional elements
(quality and properties of system operation)

Security, Reliability, Ease of Use, Maintainability etc.

Before we implement anything, we need an overall structure for the system

- What objects, architectures, files, databases, servers, services etc. should we create ?
- Only then do we know the places where requirements should be implemented
- Also helps divide up work between team members.

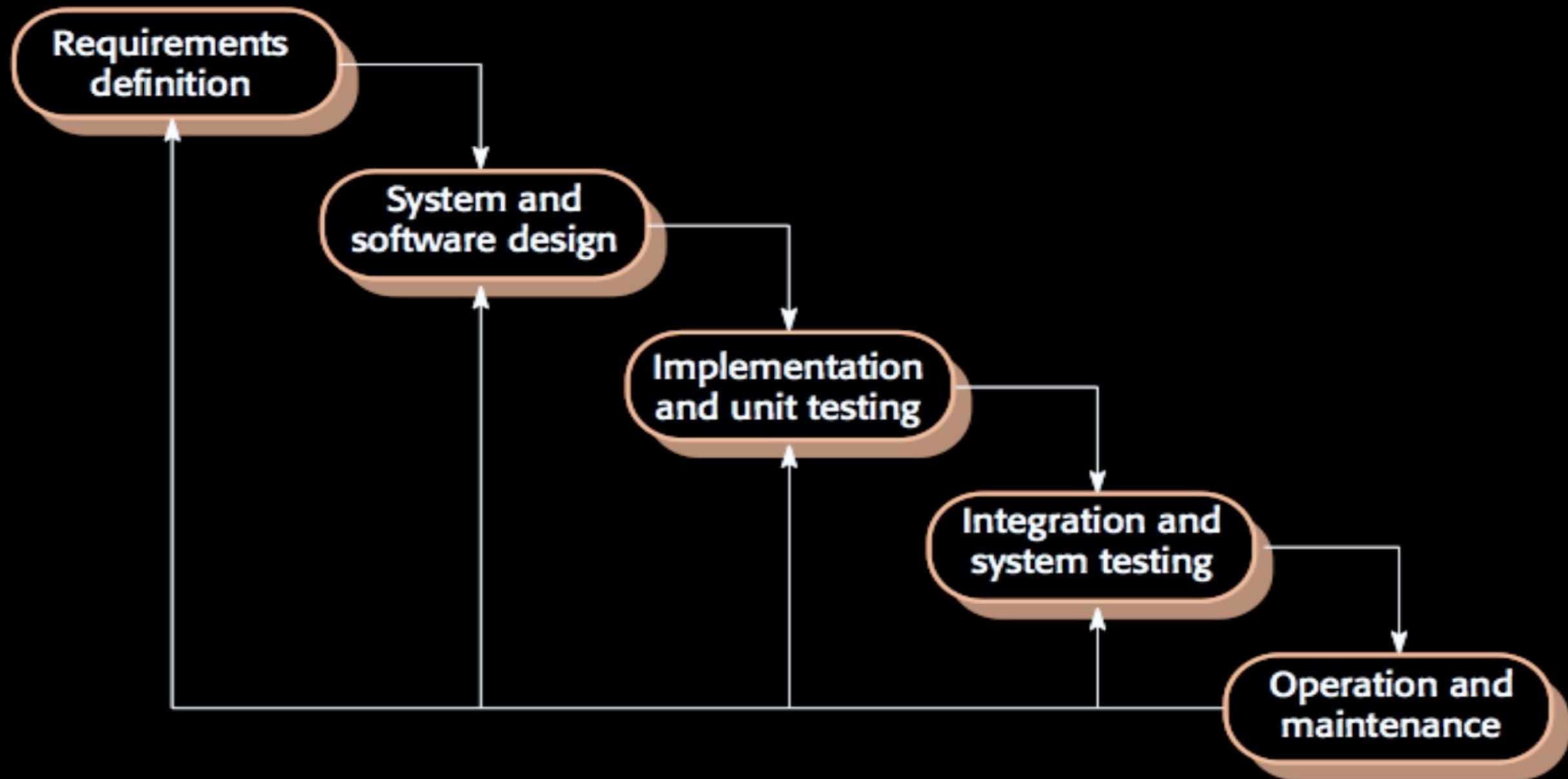
You are familiar with programming, but there is more to implementation than just “cutting code” ...

- Parallel working and code sharing
- API partitioning and “firewalling”
- Versioning, integration and config management
- Development environments and automated build
- Automated testing

Docs and training material also need writing!

Demonstrates that system meets needs of client

- Involves checking, reviewing, evaluating & testing
- Testing is the most common V & V activity
So you'll be doing plenty of that!
- User evaluation is the most undervalued
- You'll come up with excuses why you didn't do it :-D



-
- People dis waterfall model as naïve and unrealistic
 - But remember - it's a simplification & abstraction
 - A tool to help explain activities of SW development
 - Useful for undergrad Computer Science courses!
 - It's not exactly what happens in the real world,
 - But it is a good start in trying to understand it!

Agile is **NOT** a development process

It is a **philosophy** - a way of thinking

The [Agile "Manifesto"](#) promotes the following values:

- Individuals & Interactions (over processes & tools)
- Operational Software (over documentation)
- Customer Collaboration (over formal contracts)
- Responding to Change (over predefined plan)

It made old-school devs very nervous !

The values are however intended as "provocations"...

Get us to rethink and reassess our preconceptions

They don't ask us to totally disregard certain things
(such as processes, tools, documentation etc)

But rather, think about other aspects AS WELL

Agile is a development methodology
devised by (and for use by) coders!

so...

What things do coders like?

- Writing high quality code
- Refactoring code to make it more “elegant”
- Ticking things off to-do list: closure on work done
- Showing results to client to impress them

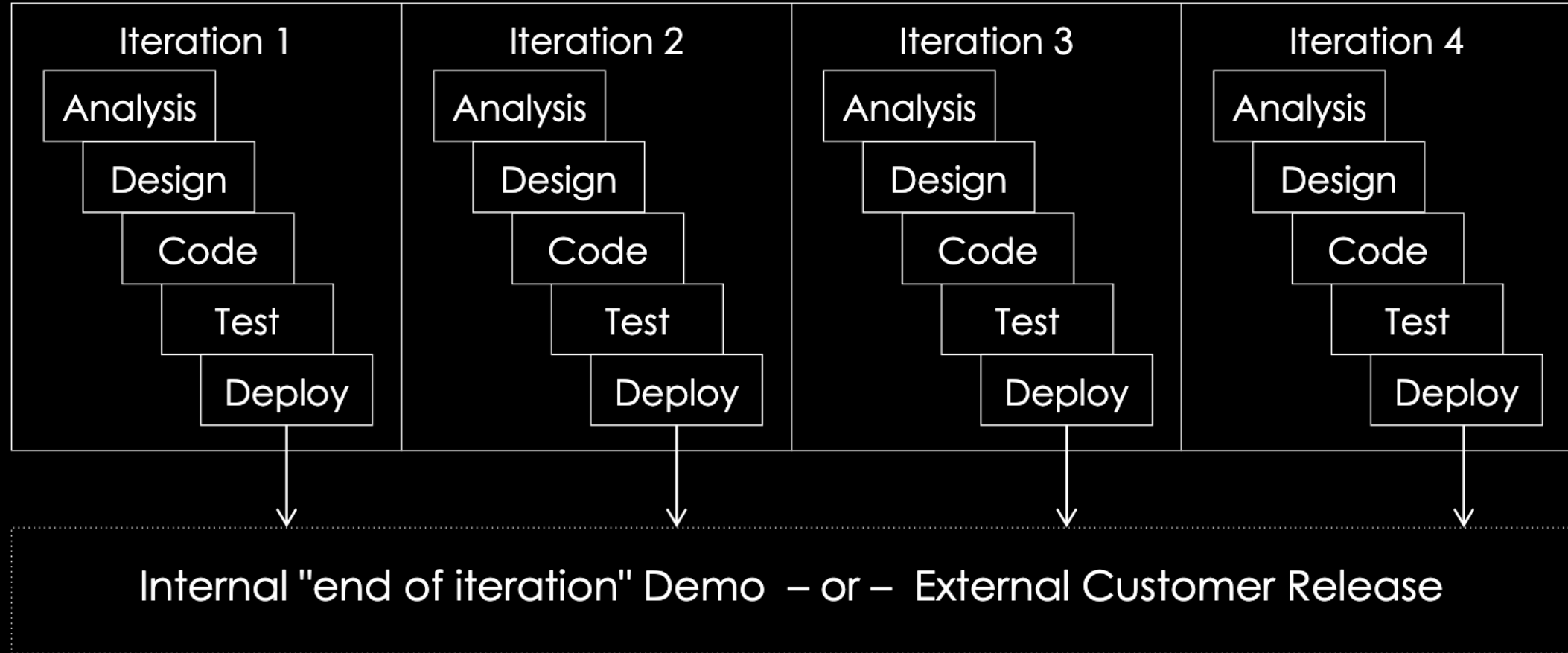
- Eliciting and recording requirements
 - Writing tedious documentation
 - Committing to a final design in advance
 - Managers telling them how to work
 - Big and immovable deadlines
-

1. Satisfying customer's needs
(with early & continuous delivery of SW)
 2. Embracing change
(even late on in the development cycle)
 3. Delivering working SW regularly
(and frequently e.g. monthly/weekly)
 4. Developer and client daily collaboration
 5. Trusting motivated individuals
 6. Ensuring face-to-face communication
-

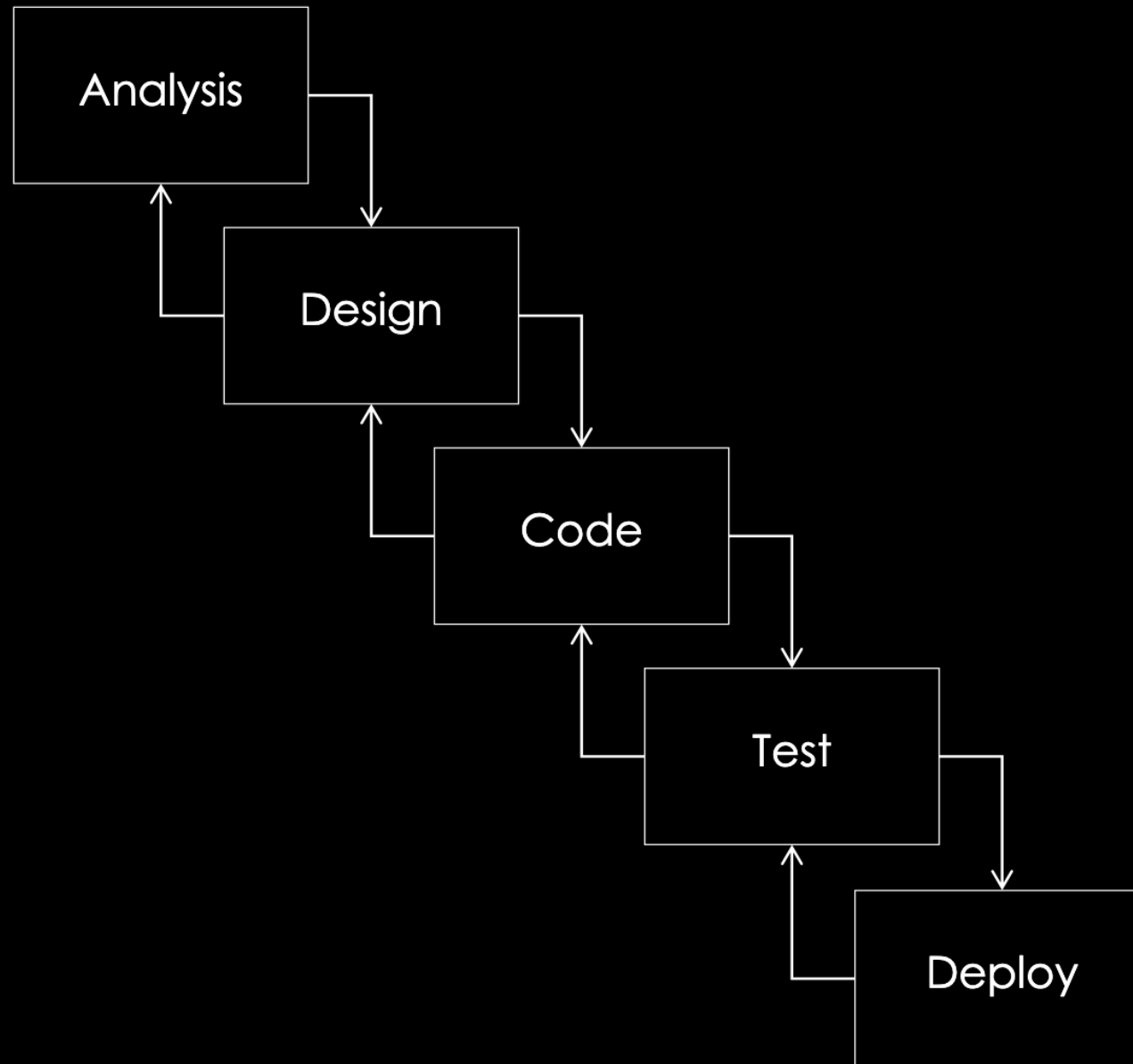
7. Measuring progress by amount of working code
8. Working at constant & sustainable pace
(not relying on heroic effort)
9. Continuous attention to technical excellence
10. Simplicity: minimise amount of (unnecessary) work
11. Rely upon self-organising teams
12. Teams regularly reflect on their own performance

- Many elements of Agile weren't new in 2001!
 - They have existed and evolved for many years
 - Agile draws them together under a single umbrella
 - If you were being really cynical you might view Agile as just a branding exercise
 - An opportunity to sell more books
 - Earn more consultancy fees :-D
-

The 'totally new' Agile development process



The 'old fashioned' generic waterfall



What does Agile look like in SEP?



shutterstock.com • 1795924660

Sprints are reoccurring (usually) 2-3 week periods in a longer project allow a team to organise and think about tasks in smaller time chunks.

Start with meetings to discuss all the work teams believe can be completed in the sprint.

This becomes tasks with definable, concrete goals which can be evaluated to determine if the task has been completed.

SEP is divided into 3 sprint equivalents:

- Minimum Viable Product phase (now until Week 9)
- Beta phase (Week 10 to Week 17)
- Final phase (Week 18 to Week 24)

BUT! You can have more sprints! This is the minimum

(A.K.A Daily Standup or Daily SCRUM)

Short meetings to keep team members updated on each individual's progress as a sprint progresses.

“Stand ups” because everyone stands up so that no one gets comfortable enough to allow the meeting to go on for too long – 15-20 minutes.

Each team member will talk about

- what they have done
- what they are *blocked* by
- what they are going to do.

Everyone gets a quick update on other's progress

Invites team members to help each other with tasks they are blocked by.

To keep this meeting on track, is the *SCRUM Master*

- Directs the agenda
- Makes sure everyone is heard
- Keeps the meeting quick and relevant

In industry: daily

In SEP: aim at once or twice a week (eg beginning and end)

Teams that do well in SEP communicate continually!

But communication doesn't need to take a lot of time

- We recommend having somewhere you're communicating
- Ideally with channels and ways to organise conversations and somewhere you can have video/audio calls
- Teams or Discord
- NOT WhatsApp! NOT where you socialise!

- Agree a set format in advance, so you come prepared
- Rotate the Scrum Master so everyone takes regular turns leading the meetings
- Ask your mentor for advice!
- Record everything you're doing/have done on the kanban board (We can't give you marks if we can't see it!)
- Use the kanban board as meeting notes/agenda

- Agree a set format in advance, so you come prepared
- Rotate the Scrum Master so everyone takes regular turns leading the meetings
- Ask your mentor for advice!
- Record everything you're doing/have done on the kanban board (We can't give you marks if we can't see it!)
- Use the kanban board as meeting notes/agenda

This is not the only meeting!

Stand-ups are NOT the only time you're meeting!

Regular sessions for working together (Wednesdays)

- Planning
 - Designing
 - Pair programming
 - Working on different tasks in the same place
 - Teaching each other about the things you've made
-

TOMORROW

- Meet your team-mates
- Arrange when/where/how often to meet
- Agree a client liaison and send out initial email
- Contact your mentor and arrange to meet them
- Start working on your Repo
- Put your first tasks on the Kanban board

Your learning on this unit

On completion of this unit, students will be able to:

1. Develop software systems within a mature and repeatable Software Development process.
2. Apply Agile Principles & Practices in the development of systems of significant scale.
3. Select and employ development support tools (as appropriate to the project being undertaken).
4. Apply basic principles of project management.
5. Work effectively within real-world development contexts and with clients.
6. Collaborate effectively within small teams.
7. Identify the presence of (and employ techniques to mitigate the impact of) project uncertainty and risk.



What is a Kanban board?

A useful concept taken from the [Kanban method](#).

A flexible 'ToDo' list structure, where issues progress through various states:

"ToDo" -> "In Progress" -> "Done"

Originally post-it notes on a whiteboard, now there are various digital tools to help you.

We are using the GitHub version, called "Projects".

Example Kanban

2023 Wall Projections Project Add status update

Main Kanban + New view

Filter by keyword or by field

Low Priority/Long-term 9	Todo 1	In Progress 0	Review and test 0	Done 242	Won't Fix 10
2023-WallProjections #252 Improve usage of PythonHandler and PythonProxy 2023-WallProjections #163 Deal with issues in the config more gracefully 2023-WallProjections #134 Add unit tests for views w/ code-behind 2023-WallProjections #161 Fix secondary window startup position on Linux 2023-WallProjections #69 Add unit tests for views	This item hasn't been started 2023-WallProjections #239 Write a blog post for the museum	This is actively being worked on	Code to be looked over and tested	This has been completed 2023-WallProjections #32 Work out a workflow for using Git collaboratively 2023-WallProjections #192 Create a GH release for the Beta release 2023-WallProjections #116 Add animations and transitions 2023-WallProjections #222 Add more logging to python 2023-WallProjections #26	2023-WallProjections #42 Research Java cross-compilation for Raspberry Pi 2023-WallProjections #49 Continue experimenting with mediapipe and hand-tracking 2023-WallProjections #95 Upgrade mediapipe to new solutions 2023-WallProjections #31 Gantt chart 2023-WallProjections #48

Where do you start?

The process of establishing the features that the client requires from a system

and the constraints under which such features must operate

The requirements are the recorded description of the system features and the constraints that are generated during this process.

We will talk to you more about this in the coming weeks, but the task starts with your first client meeting

We must find out what to build before we build it!

Common trap: to **think** we know what the client wants and blaze ahead building it....

...only to realise later we were **wrong**!

Having wasted time, effort and money (grades)

How do we actually find the requirements?

We don't just know them intuitively
(a rookie developer mistake is to think that you can!)

We have to actually make an effort to find them

They can be elicited using a variety of techniques...

We'll talk about this more in Week 3

Organisations often have a lot of documentation:
process documents, regulations, manuals, etc

Ask your client for theirs!

If we can get hold of them and pick them apart, we can understand how the organisation operates and then specify requirements to support that operation

BUT!

Be aware that people don't always follow official regulation!

Being able to identify work-arounds being used is essential!

An obvious way to find out about an organisation is to talk to people (workers, managers, customers)

This may be an informal chat over coffee
Or a formal, planned, structured interview

Be open-minded and avoid pre-conceived notions.

You can prompt the interviewee to get discussions going by using a 'springboard' question...

BUT do try to listen to their answers!

Another way to identify requirements is by observation
Watch the 'lived' practices within an organisation

Social and organisational factors may be observed

Some social scientists (ethnographers) are explicitly trained in the observation and analysis of work

But it is possible to undertake our own studies

Existing systems encapsulate work practices

- If we analyse/deconstruct them, we gain insight
- Reverse engineer systems to derive requirements
- **BUT....** This assumes existing systems are good
- If they are that good, why build something new?

Stakeholders will express their needs in their own words

- This may be in the technical jargon of their field
 - They may struggle to articulate what they want
 - Or use concepts that only make sense to them!
 - Different stakeholders often have conflicting needs - how can you reconcile these?
 - Stakeholders frequently don't know what they want
 - Often they don't even know what they currently do (or at least can't explain)
-

Problems with Requirements Analysis

- Organisational and political factors may be present
- Development is not just a technical challenge, but a human one
- Requirements may change during the analysis process
- New stakeholders emerge or the environment changes
- Different people in the organisation may contradict each other
- People might not know what's possible or not

How to solve these problems

- If you're unclear about anything the client says, ask them!
- Never pretend you understand things you don't – the clients will always prefer you stop and check
- Send them emails after the meeting to confirm what you think you were told
- If you're unclear what things mean, or need help, talk to your mentor and come to the Drop-Ins on Fridays for help

Use your first meeting to start your Requirement Engineering Process

- Ask the client if they can share information about the existing systems and their organisation
- Ask if you can plan another meeting specifically for Requirement Engineering – don't assume this is something you can do in your first meeting
- Ask if you can come and observe their work – but don't be surprised/upset if they say no (they are very busy)

Who are your **Stakeholders**?

Anyone who might be **impacted** by the system
and who might **impact** the system
even if they don't directly interact with the system.

- End users
- Organisation and system managers
- Organisation and system owners
- General public who are affected by the system - this can be directly (e.g. people who want to use Community Transport) and indirectly (businesses/services/people expecting others to use Community Transport)
- Legislators (for example, with an interest in GDPR and data privacy)

Once we have identified a set of stakeholders for a project, we can start to think about what these individuals and organisations might need from a system. We do this through identifying **User Stories**.

A popular way to record user stories is:

As a [type of user], I want to [some goal], so that [some reason]

Or to put it more technically:

As a [persona], I want to [action], so that [benefit].

For example:

As a chef, I want to be able to identify how long it takes to cook meals, so that I have time to cook and deliver and order in a reasonable time.

As a diner, I want to be able to order a meal, so that I can eat it and no longer be hungry.

As a diner, I want to try a dish I've never tasted before, so that I can experience something new and taste new flavour combinations.

Once you have identified your user stories, you next identify the "**flow**" steps.

The "**basic flow**" is the normal or typical sequence of steps involved in achieving the goal.

For example, in a restaurant, when a customer wants a meal, the customer flow steps are:

- Consider menu
- Pick starter
- Pick main course
- Attract waiter's attention
- Place order
- Waiter passes order to chef
- Chef confirms order with waiter

However, be aware that interaction does not always follow the basic flow:

Alternative flow:

Another, equally valid, but perhaps less common sequence of steps
(sometimes you choose the main course before the starter!)

Exceptional flow:

Unusual path to cope with strange or undesirable behaviour
(you see the price of the food and slip away quietly!)

Before you meet your client

In your team, think about **who your client's stakeholders might be**, and write a few user stories with flow steps, and add them to your README.

You won't yet have much detailed knowledge regarding your project, so take a best guess.

When you meet with your client, you can use this as a starting point for discussion.

User Stories can and should evolve over time as you gain more knowledge and insight into your application domain.

What else will you do tomorrow?

1. Meet your team!
 2. Plan where/when/how you will meet
 3. Start your Kanban Board
 4. Agree a client liaison and email the client to arrange your first meeting
 5. Contact your mentor and arrange the first meeting
 6. Start your README:
 - * Project description
 - * Stakeholders
 - * User stories
 7. Think about your Contract with your client
 8. Think about Data Protection and Ethics for your projects
-